Unit -2 Intelligent Agents

AGENTS AND ENVIRONMENTS

What Is an Agent?

An **agent** is anything that:

1. **Receives information** from its surroundings (using **sensors**)
2. **Take actions** based on that information (using **actuators**)

An agent **senses the world**, **remembers** what it has sensed, and **takes actions** based on that information.

Sensors = How the agent gets information

Sensors help the agent *notice* what is happening around it.

- **Human** → eyes, ears, nose, skin
- **Robot** → camera, microphone, infrared sensors
- **Software program** → keyboard input, files, network messages

Actuators = How the agent acts

Actuators help the agent *do something* in response.

- **Human** → hands, legs, mouth
- **Robot** → motors, wheels, robotic arms
- **Software program** → display text, save files, send data

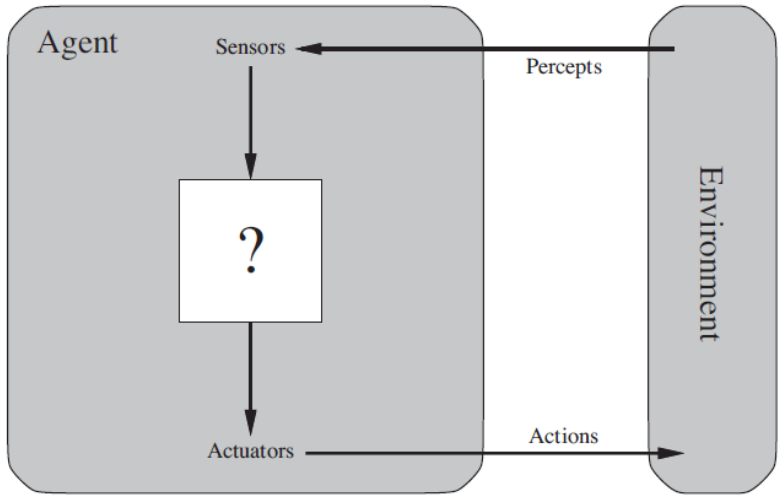

Figure 2.1 Agents interact with environments through sensors and actuators.

What is a Percept?

A **percept** is **what the agent senses at one moment**.

Example:

- You see a red traffic light → that sight is a percept.
- A robot detects an obstacle → that detection is a percept.

What is a Percept Sequence?

A **percept sequence** is **the complete history of everything the agent has sensed so far**.

Example:

- First you saw a green light
- Then you saw a yellow light
- Then you saw a red light

All of those together form your percept sequence.

Agent Function

Mathematically, an **agent's behaviour** can be described by something called an:

Agent Function

An **agent function** is a rule that:

Takes everything the agent has sensed so far (percept sequence)
➜ and tells the agent what action to take next.

So basically:

Percept sequence → Action

We can imagine making a big table like this:

What the agent has sensed	**Action, it takes**
Dirty square A	Suck
Clean square A	Move Right
Dirty square B	Suck
Clean square B	Move Left

This table describes **how the agent behaves**.

But in real life, this table would be HUGE — possibly infinite — because there are so many possible percept histories.

Agent Function vs Agent Program

These two are different:

Agent Function

- Abstract (mathematical idea i.e. existing only as an idea, not as a physical thing)
- Just describes **what action should happen** for each percept sequence
- Like a rulebook

Agent Program

- The **actual code running** on a computer or robot
- **Implements** (follows) the agent function

- Like the real machine **following** the **rulebook**

Think of it like:

- **Agent function** = recipe
- **Agent program** = cook using the recipe

Example: Vacuum Cleaner World

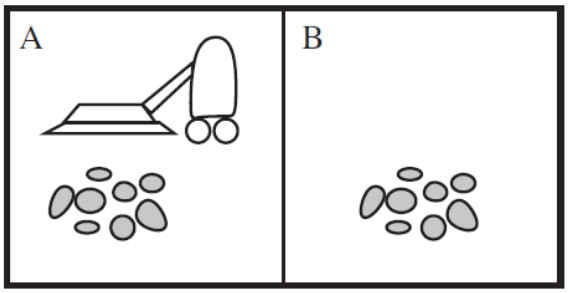

Figure 2.2 A vacuum-cleaner world with just two locations.

Imagine a very simple world:

- Only **two locations**: A and B
- The vacuum can sense:
  - Where it is (A or B)
  - Whether there is dirt
- The vacuum can:
  - Move Left
  - Move Right
  - Suck
  - Do Nothing

A Very Simple Agent Rule

The rule (agent function) is:

If the square is dirty → Suck
If the square is clean → Move to the other square

Percept sequence	Action
$[A, Clean]$	$Right$
$[A, Dirty]$	$Suck$
$[B, Clean]$	$Left$
$[B, Dirty]$	$Suck$
$[A, Clean], [A, Clean]$	$Right$
$[A, Clean], [A, Dirty]$	$Suck$
$\vdots$	$\vdots$
$[A, Clean], [A, Clean], [A, Clean]$	$Right$
$[A, Clean], [A, Clean], [A, Dirty]$	$Suck$
$\vdots$	$\vdots$

Figure 2.3 Partial tabulation of a simple agent function for the vacuum-cleaner world shown in Figure 2.2.

What **Makes** an **Agent "Good"?**

Which **actions make** the **agent smart?**
Which **make it dumb?**

That **depends** on **how well the agent achieves its goal**.

For example:

- A vacuum that keeps moving but never cleans = bad agent
- A vacuum that cleans efficiently = good agent

Are All Things Agents?

Technically, almost anything that:

- Receives input
- Produces output

could be called an agent.

Even a calculator could be called an agent:

- Input: "2 + 2 ="
- Output: "4"

But calling it an agent doesn't really help us understand it better.

Why AI Uses the Agent Idea

AI focuses on systems that:

- Have significant **computing power**
- Must make **decisions**
- Operate in **complex** environments

So, AI studies **intelligent agents** — systems that must think before acting.

The concept of rationality

Definition

A rational agent does **the action that maximizes expected performance**, based on:

- The **percept sequence** it has observed
- Its built-in **knowledge**
- The available **actions**
- The defined **performance measures**

So "**doing the right thing**" means:

Choosing the **action** that **maximizes** the expected value of the **performance** measure.

The performance measure must **evaluate** the **environment state**, **not the agent's internal beliefs.**

Why?

Because otherwise an agent could:

- Fool itself
- Redefine success internally
- Claim perfect performance without actually improving the world

Example:
If success were defined as *"**the agent feels satisfied**"*, then a rational agent would simply **modify itself** to feel satisfied all the time.

That would be rational — but useless.

The vacuum example highlights a famous AI principle:

A rational **agent will optimize exactly what you measure** — even if that's not what you intended.

If performance = "amount of dirt cleaned,"
the agent may:

- Clean dirt
- Dump it again
- Re-clean it
- Repeat forever

It's rational — but not aligned with the real goal.

This teaches:

- **Performance measures** must **reflect** the **actual desired outcome**.

- They must **not reward** behaviour **shortcuts**.

This **idea is extremely important today** in:

- AI alignment

  AI Alignment means designing AI systems so that their goals, decisions, and behaviours are **aligned with human values and intentions**.

- Reward hacking

  Reward hacking happens when an intelligent agent finds a way to **maximize its reward (performance measure)** in an unintended or undesirable way — without actually achieving the real goal.

- Reinforcement learning
- Economics and incentive design

Rationality Is Relative to the Performance Measure

Different performance measures lead to **different rational behaviours**.

Example:

- Agent A: cleans moderately all the time
- Agent B: cleans perfectly, then takes long breaks

If both achieve the same average cleanliness:

- Which is better?

That depends entirely on **how you define the performance measure**.

Omniscience, learning, and autonomy

Rationality ≠ Omniscience

An **omniscient agent**:

- Knows the ***actual* outcome** of its actions
- Knows the **future** perfectly
- Always chooses the **action** that turns out **best**

But omniscience is impossible in reality.

Information Gathering -

Rational agents should:

- Perform actions that **improve their knowledge**
- Not just act immediately

Learning -

A rational agent must:

- **Learn** from experience
- **Update** its knowledge
- **Improve** performance over time

If it doesn't learn:

- It becomes fragile
- It fails in slightly changed environments

Autonomy -

The **agent's behaviour depends on its own experience**, not only on built-in knowledge.

A highly autonomous agent:

- **Learns** from percepts
- **Adapts** to environment
- **Compensates** for incorrect prior knowledge

A **non-autonomous agent**:

- Depends entirely on designer assumptions
- Fails when assumptions are violated

The nature of environment

To **build rational agents** -

First, we must **think about task environments**, which are essentially the "**problems**" to which rational agents are the "solutions."

1. Specifying the task environment

To **specify** the **performance measure**, the **environment**, and the **agent's actuators** and **sensors**.

This is called the **PEAS** (**Performance**, **Environment**, **Actuators**, **Sensors**) description.

So,

Task Environment = Problem to Be Solved

- A task environment defines the problem.
- A rational agent is the solution to that problem.

Before designing an agent, you **must clearly define the task** environment.

Let us consider an **example**: **an automated taxi driver**.

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits	Roads, other traffic, pedestrians, customers	Steering, accelerator, brake, signal, horn, display	Cameras, sonar, speedometer, GPS, odometer, accelerometer, engine sensors, keyboard

Figure 2.4 PEAS description of the task environment for an automated taxi.

First, what is the **performance measure** to which we would like our automated driver to aspire?

- Getting to the **correct destination**;
- **Minimizing fuel consumption** and wear and tear;
- **Minimizing** the **trip time or cost**;
- **Minimizing violations** of **traffic laws** and disturbances to other drivers;
- Maximizing **safety** and passenger comfort;
- Maximizing **profits**.

What is the **driving environment** that the taxi will face?

- Any taxi driver must deal with a **variety of roads**, ranging from rural lanes and urban alleys to 12-lane freeways.
- The roads contain other traffic, pedestrians, stray animals, road works, police cars, puddles, and potholes.
- The taxi must also interact with potential and actual passengers.
- It could always be driving on the right, or we might want it to be flexible enough to drive on the left when in Britain or Japan. Obviously, the more restricted the environment, the easier the design problem.

The **actuators** for an automated taxi include those available to a human driver:

- **control** over the engine through the **accelerator** and control over **steering** and **braking**.
- it will need **output** to a **display screen** or **voice synthesizer** to talk back to the passengers, and perhaps some way to communicate with other vehicles, politely or otherwise.

The basic **sensors** for the taxi will include –

- one or more controllable **video cameras** so that it can see the road;
- it might augment these with **infrared** or **sonar sensors** to detect distances to other cars and obstacles.
- To avoid speeding tickets, the taxi should have a **speedometer**, and to control the vehicle properly, especially on curves, it should have an accelerometer.
- To **determine the mechanical state of the vehicle**, it will need the usual array of **engine**, **fuel**, and **electrical system sensors**.
- Like many human drivers, it might want a **global positioning system** (GPS) so that it doesn't get lost.
- Finally, it will need a **keyboard or microphone** for the passenger to request a destination.

Properties of task environments

Fully observable vs. partially observable

Fully Observable Environment

A task environment is **fully observable** if the agent's sensors provide **access** to the **complete state of the environment** at every moment.

What this means:

- The agent can see **everything that matters** for decision-making.
- No important information is hidden.
- The agent does **not need memory** to track past states.

- It can **decide** what to do based only on the current perception.

Example:

- A chess-playing AI can see the entire chessboard.
- There is no hidden information.
- The AI always knows the exact state of the game.

Partially Observable Environment

A task environment is **partially observable** if the agent **cannot access the complete state** of the environment.

This happens when:

- Sensors are **noisy or inaccurate**
- Some parts of the environment are **hidden**
- Important information is **missing**

Examples:

- A vacuum cleaner robot that only senses dirt in its current square (it doesn't know about other squares).
- An automated taxi that cannot see what other drivers are planning to do.
- Poker (players cannot see opponents' cards).

Unobservable Environment

If the agent has **no sensors at all**, the environment is **unobservable**.

This sounds hopeless, but:

- If the environment is predictable,
- And the agent knows how the world works,

it may still achieve its goal (for example, by following a fixed sequence of actions).

Single agent vs. multiagent

Single-Agent Environment

A single-agent environment is one where only **one agent's actions affect the outcome**.

- There are no other agents trying to maximize their own goals.
- The environment may change, but not because of intelligent opponents.
- The agent only needs to consider **physics, randomness, or fixed rules**, not other decision-makers.

Example:

- Solving a crossword puzzle alone.
- A robot navigating an empty warehouse.

In these cases, success depends only on the agent's own decisions.

Multiagent Environment

A multiagent environment contains **more than one agent**, where:

- Each **agent** may have its **own goals**.
- One agent's success may depend on others' actions.
- Other agents may act strategically.

Competitive Multiagent Environment

- In a **competitive environment**, agents have **conflicting goals**.
- chess is a competitive multiagent environment.
- In the **taxi-driving environment**, on the other hand, avoiding collisions maximizes the performance measure of all agents, so it is a **partially cooperative multiagent** environment.
- It is also **partially competitive** because, for example, only one car can occupy a parking space.

Example: **Chess**

- One player's win = the other player's loss.
- Each player tries to **maximize their own performance measure**.
- Rational play requires predicting the opponent's strategy.
- Randomized strategies can sometimes be useful to avoid being predictable.

Deterministic vs. Stochastic

- If the next state of the environment is completely determined by the current state and the action executed by the agent, then we say the environment is **deterministic**; otherwise, it is **stochastic**.
- In principle, an agent need not worry about uncertainty in a **fully observable**, **deterministic** environment.
- If the environment is **partially observable**, however, then it could *appear* to be **stochastic**.

Uncertain vs. Nondeterministic

- Environment is **uncertain** if it is **not fully observable** or not deterministic.
- Stochastic generally implies that **uncertainty** about outcomes is **quantified** in terms of **probabilities**;

- **Nondeterministic** environment is one in which actions are **characterized** by their *possible* **outcomes**, but **no probabilities** are attached to them.
- Nondeterministic environment descriptions are usually associated with performance measures that require the **agent to succeed for *all possible* outcomes** of its actions.

Episodic vs. Sequential

- In an **episodic** task environment, the agent's **experience is divided into atomic episodes.**
- In each episode the agent receives a percept and then performs a single action.
- Crucially, the **next episode does not depend** on the **actions taken in previous episodes.**
- Many classification tasks are episodic.
- For **example**, an agent that has to spot defective parts on an assembly line bases each decision on the current part, regardless of previous decisions; moreover, the current decision does not affect whether the next part is defective.
- In **sequential environments**, on the other hand, the current decision could **affect all future decisions.**
- Chess and taxi driving are sequential: in both cases, short-term actions can have long-term consequences.

Static vs. Dynamic (vs. Semidynamic)

- If the environment can change **while an agent is deliberating**, then we say the **environment is dynamic** for that agent; otherwise, it is static.
- **Static environments** are **easy to deal** with because the agent need not keep looking at the world while it is deciding on an action, nor need it worry about the passage of time.
- **Dynamic environments**, on the other hand, are continuously asking the agent what it wants to do; if it hasn't decided yet, that counts as deciding to do nothing.
- If the **environment itself does not change** with the passage of time but the **agent's performance score does**, then we say the environment is **semi dynamic**.
- **Example** –
    - Taxi driving is clearly dynamic: the other cars and the taxi itself keep moving while the driving algorithm dithers about what to do next.
    - Chess, when played with a clock, is semi dynamic.
    - Crossword puzzles are static.

Discrete vs. Continuous

Discrete: The environment has distinct states and time steps.

- **actions** available are distinct and separate.

- The environment changes in **separate steps**, not continuously.

Example: A turn-based board game like checkers.

Continuous: The environment changes smoothly over time.

- The **state space** contains real numbers (not just countable positions).
- The **action space** allows infinitely many possible values.

Example: Self driving car, where speed and steering are continuously variable

Known vs. Unknown

Known: The agent knows the rules and consequences of actions within the environment.

Example: A robot programmed to assemble parts in a factory with known procedures.

Unknown: The agent has to learn the effects of its actions through exploration.

Example: An agent exploring a new video game for the first time, where the agent learns the rules and controls as it plays.

Environment Types

	Solitaire	Backgammon	E-shopping	Taxi
Observable?				
Deterministic?				
Episodic?				
Static?				
Discrete?				
Single-agent?				

Environment Types

	Solitaire	Backgammon	E-shopping	Taxi
Observable?	Yes	Yes	No	No
Deterministic?	Yes	No	Semi	No
Episodic?	No	No	No	No
Static?	Yes	Semi	Semi	No
Discrete?	Yes	Yes	Yes	No
Single-agent?	Yes	No	Yes/No	No

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Interactive English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

Figure 2.6 Examples of task environments and their characteristics.